

Programming in Python

Maulana Hafidz Ismail

31 Oct 2025

Contents

1	Memulai dengan Python	1
1.1	Tentang Python	1
1.2	Sejarah Pemrograman	1
1.3	Dasar-dasar Pemrograman	1
1.4	Apa itu Pemrograman?	1
1.5	Manfaat Belajar Python	2
1.6	Aplikasi Python	2
1.7	Menjalankan Python di Command Line	2
1.8	Sintaksis Python dan Pentingnya Spasi	2
1.9	Memberi Komentar pada Kode	3
1.9.1	Single-line comments	4
1.9.2	Multi-line comments	4
1.9.3	Inline comments	4
1.10	Pengertian Variabel	4
1.11	Penamaan Variabel	4
1.12	Mengubah dan Menghapus Variabel	5
1.13	Deklarasi Beberapa Variabel	5
1.14	Tipe Data dalam Python	5
1.15	Jenis Tipe Data Utama	5
1.16	Menentukan Tipe Data	6
1.17	Deklarasi Strings	6
1.18	Mengakses dan Mengubah String	7
1.19	Fungsi dan Operasi pada Strings	7
1.20	Contoh Penggunaan	7
1.21	Cheatsheets Tipe Data dan Fungsi	7
1.21.1	Tipe Data	7
1.21.2	Flow Control	7
1.21.3	Indentasi	8
1.21.4	Komentar	8
1.21.5	Built-in Functions	9
1.21.6	Membuat Fungsi	10
1.22	Apa itu Typecasting?	10
1.23	Implicit Type Conversion	10
1.24	Explicit Type Conversion	10
1.25	Fungsi Typecasting Lainnya	11
1.26	Fungsi Input	11
1.27	Fungsi Print	11
1.28	Parameter Tambahan untuk Print	11
1.29	Contoh Penggunaan Input dan Print	11
1.30	Penanganan Tipe Data	12
1.31	Penggabungan String	12
1.32	Melihat Lebih Dalam ke Type Casting	12
1.33	Operator Matematika	14
1.34	Operator Logika	15

1.35	Contoh Penggunaan Operator dalam Pernyataan Kondisional . .	15
1.36	Apa itu Control Flow?	15
1.37	Conditional Statements	16
1.38	Contoh Penggunaan Control Flow	16
1.39	Penggunaan Match Statement	16
1.40	Perbandingan dengan if Statement	17
1.41	Fitur Penting dari Match Statement	17
2	Dasar Programming dengan Python	18
3	Paradigma Pemrograman	19
4	Modul, Paket, Pustaka, and Alat-alat	20

1 Memulai dengan Python

1.1 Tentang Python

Python dinamai setelah acara TV "Monty Python's Flying Circus" dan bukan dari ular piton. Nama ini dipilih karena dianggap singkat dan misterius. Python digunakan dalam aplikasi yang sering kita gunakan, seperti Instagram, Facebook, Google, dan Spotify. Contoh penggunaan Python dalam machine learning termasuk TensorFlow, yang digunakan oleh Airbnb untuk mengklasifikasikan gambar dan oleh perusahaan kesehatan untuk mengklasifikasikan data MRI. Python dianggap mudah digunakan dan sederhana, dengan banyak pustaka (libraries) yang mendukung pengembangan. Memungkinkan pengembang untuk membuat fitur dengan cepat dan melihat hasilnya lebih cepat.

1.2 Sejarah Pemrograman

- Charles Babbage, pada tahun 1822, menciptakan difference engine untuk mengurangi kesalahan manusia dalam perhitungan.
- Babbage juga mengembangkan analytical engine, yang dianggap sebagai dasar dari komputer modern.
- Teman Babbage, Ada Lovelace, menulis dokumen yang menjelaskan bagaimana analytical engine dapat melakukan urutan perhitungan, mirip dengan apa yang dilakukan oleh program komputer.

1.3 Dasar-dasar Pemrograman

- Komputer hanya memahami binary code, yang terdiri dari dua digit: 0 dan 1. Binary Code: Sistem bilangan yang hanya menggunakan dua angka, 0 (off) dan 1 (on).
- Setiap program yang ditulis harus dikonversi menjadi machine code (kode mesin) agar dapat dipahami oleh komputer. Contoh konversi: Decimal 1 menjadi Binary 1, Decimal 2 menjadi Binary 10.
- Komputer menggunakan transistors untuk merepresentasikan binary code. Transistor: Komponen elektronik yang berfungsi sebagai saklar untuk mengontrol aliran listrik.

1.4 Apa itu Pemrograman?

Pemrograman adalah kemampuan untuk memberikan serangkaian instruksi kepada komputer dalam bahasa yang dapat dipahami.

Pemrograman adalah keterampilan yang dapat diasah dengan latihan. Semakin sering berlatih, semakin baik kemampuan pemrograman seseorang.

Pemrograman juga merupakan keterampilan kreatif, karena ada banyak cara untuk menyelesaikan masalah dengan menulis program.

1.5 Manfaat Belajar Python

- Mudah Dipelajari: Python memiliki sintaks yang mirip dengan bahasa Inggris, sehingga lebih mudah dibaca dan dipahami.
- Produktivitas Tinggi: Dengan Python, pengembang dapat menyelesaikan proyek lebih cepat karena sintaks yang sederhana dan memerlukan lebih sedikit kode dibandingkan dengan bahasa lain seperti C atau Java.

1.6 Aplikasi Python

- Berbagai Platform: Python dapat digunakan di Windows, Mac, dan Linux.
- Bidang Penggunaan: Python banyak digunakan dalam pengembangan web, kecerdasan buatan, machine learning, analisis data, dan aplikasi pemrograman lainnya.

1.7 Menjalankan Python di Command Line

- Python Shell:

Ini adalah cara pertama untuk menjalankan program Python. Shell ini berguna untuk menjalankan dan menguji skrip kecil tanpa perlu membuat file baru dengan ekstensi .PY. Anda dapat mengetikkan potongan kode langsung di shell.

Cukup ketik `python` di command line untuk membuka Python shell, lalu masukkan kode yang ingin dijalankan.

- Menjalankan File Python

Cara kedua adalah menjalankan file Python langsung dari command line. Setiap file dengan ekstensi .PY dapat dijalankan dengan perintah tertentu.

Ketik `python nama_file.py` di command line untuk menjalankan file tersebut.

1.8 Sintaksis Python dan Pentingnya Spasi

Menggunakan `print("Hello")` untuk menghasilkan teks. `print` adalah fungsi yang digunakan untuk menampilkan output ke layar.

```
print("Hello") print("World")
```

Jika mencoba menulis dua pernyataan `print` dalam satu baris tanpa pemisah, akan muncul error `SyntaxError: invalid syntax`. `SyntaxError` adalah kesalahan yang terjadi ketika kode tidak mengikuti aturan sintaks yang benar.

Mengatasi Error:

- Metode Pertama: Memindahkan pernyataan `print` kedua ke baris baru.

```
print("Hello")
print("World")
```

- Metode Kedua: Memisahkan pernyataan dengan titik koma.

```
print("Hello"); print("World")
```

Whitespace adalah karakter kosong (spasi, tab, newline) yang tidak terlihat tetapi mempengaruhi struktur kode. Menambahkan spasi di sekitar operator tidak menyebabkan masalah, tetapi menambahkan baris baru bisa menyebabkan error. Contohnya seperti ini:

```
#any ammount of whitespace on a single line is ok
x      =      1      +      2
```

Dan berikut tidak benar:

```
x = 1
+ 2
```

Indentasi adalah penggunaan spasi atau tab untuk mengatur blok kode, yang menunjukkan hierarki dan struktur. Penting untuk struktur kontrol seperti if statement.

```
if name == "John":
    print(name)
```

Jika indentasi dihapus, akan muncul error `IndentationError: expected an indented block`.

1.9 Memberi Komentar pada Kode

Menambahkan komentar ke kode tidak hanya membantu Anda sebagai pengembang, tetapi juga membantu anggota tim lainnya. Komentar sangat berguna untuk menyegarkan ingatan Anda tentang kode yang mungkin telah Anda tulis berbulan-bulan lalu dan Anda mungkin lupa apa tujuannya. Ada beberapa alasan mengapa Anda perlu menambahkan komentar ke berkas kode. Alasannya antara lain:

- Menjelaskan tujuan kode.
- Memberitahu pengembang bahwa kode tersebut sudah usang.
- Menambahkan komentar TODO untuk pekerjaan yang akan diselesaikan nanti.

Berikut adalah contoh berbagai jenis komentar yang dapat digunakan.

1.9.1 Single-line comments

Penggunaan simbol `#` memberi tahu Python untuk mengabaikan semua yang ada setelah titik ini hingga akhir baris saat ini.

```
# Don't try to Run Me, I'm a comment  
x = 10
```

1.9.2 Multi-line comments

Tanda Kutip Tiga (''' atau '''): Python menggunakan tanda kutip tunggal atau ganda tiga kali untuk membuat komentar multi-baris. Tanda kutip ini biasanya digunakan untuk docstring atau komentar yang mencakup beberapa baris.

1.9.3 Inline comments

Simbol `#` memberi tahu Python untuk mengabaikan semua yang ada setelah titik ini hingga akhir baris saat ini. Hal ini memungkinkan komentar ditempatkan di dalam kode itu sendiri. Komentar sebaris sebaiknya digunakan untuk memberikan informasi penting tentang kode yang ditanganinya.

```
x = 1 # Resetting buffer size
```

Ingatlah untuk selalu memiliki alasan untuk berkomentar, komentar harus memberikan informasi kepada pembaca dan tidak mengganggu.

1.10 Pengertian Variabel

Variabel adalah tempat untuk menyimpan data yang dapat berubah. Dalam Python, Anda dapat mendeklarasikan variabel dengan memberikan nama dan nilai.

Contoh sintaks: `X = 10` (di mana `X` adalah nama variabel dan `10` adalah nilainya).

1.11 Penamaan Variabel

Penting untuk memberikan nama yang bermakna agar mudah dipahami oleh programmer lain.

Dua konvensi penamaan yang umum digunakan:

- **camelCase**: Huruf pertama dari kata pertama kecil, dan huruf pertama dari setiap kata berikutnya besar. Contoh: `myName`.
- **snake_case**: Semua huruf kecil dengan garis bawah sebagai pemisah. Contoh: `my_name`.

1.12 Mengubah dan Menghapus Variabel

Anda dapat mengubah nilai variabel yang sudah dideklarasikan dengan cara mendeklarasikannya kembali.

```
a = 10
print(a) # Output: 10
a = 5
print(a) # Output: 5
```

Untuk menghapus variabel, gunakan perintah del.

```
del a
print(a) # Akan menghasilkan error: NameError: name 'a' is
        not defined
```

1.13 Deklarasi Beberapa Variabel

Anda dapat mendeklarasikan beberapa variabel sekaligus.

```
a, b, c = 1, 2, 3
print(a, b, c) # Output: 1 2 3
```

1.14 Tipe Data dalam Python

Tipe data adalah atribut yang terkait dengan data yang memberi tahu sistem komputer bagaimana menginterpretasikan nilainya.

Tipe Data memastikan data dikumpulkan dalam format yang diinginkan dan nilai setiap properti sesuai harapan.

1.15 Jenis Tipe Data Utama

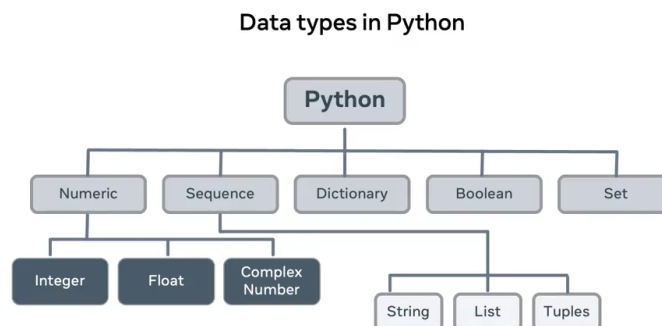


Figure 1: Pohon Tipe Data

- Numeric: Tipe data ini mencakup:

- Integer: Angka bulat tanpa desimal, misalnya 10 atau -10.
- Float: Angka dengan desimal, misalnya 10.5 atau 6.7.
- Complex: Angka kompleks yang terdiri dari bagian nyata dan imajiner, misalnya $a = 10 + 10j$.
- Sequence: Tipe data yang berisi satu atau lebih tipe dalam urutan yang teratur, termasuk:
 - String: Sekuens karakter yang diapit oleh tanda kutip, misalnya "Hello".
 - List: Sekuens yang dapat berisi berbagai tipe, diapit oleh tanda kurung siku, misalnya [1, 2, 3].
 - Tuple: Mirip dengan list, tetapi tidak dapat diubah (immutable), diapit oleh tanda kurung, misalnya (1, 2, 3).
- Dictionary: Menyimpan data dalam struktur objek key-value. Contoh:

```
ed = {'a': 22, 'b': 44.4}
```

- Boolean: Tipe data yang hanya memiliki dua nilai, yaitu True atau False. Contoh:

```
is_valid = True
```

- Set: Koleksi tidak teratur dari nilai yang tidak diulang. Contoh:

```
example_set = {1, 2, 3, 4}
```

1.16 Menentukan Tipe Data

Fungsi `type()`: Digunakan untuk menentukan tipe data dari variabel. Contoh:

```
a = 10
print(type(a)) # Output: <class 'int'>
```

1.17 Deklarasi Strings

String adalah sekuens karakter yang dikelilingi oleh tanda kutip tunggal (') atau ganda ("). Contoh sintaks: `a = 'hello'` atau `b = "hello"`.

Encoding adalah proses mengubah karakter menjadi bentuk yang dapat dipahami komputer. Python menggunakan `Unicode` untuk encoding.

1.18 Mengakses dan Mengubah String

- Indexing: Setiap karakter dalam string dapat diakses menggunakan indeks. Python menggunakan zero-indexing, artinya indeks dimulai dari 0. Contoh: `name[0]` mengakses karakter pertama.
- Mengubah Nilai: Anda dapat mengubah nilai string dengan cara menetapkan nilai baru ke variabel. Contoh: `name = 'Paul'`.

1.19 Fungsi dan Operasi pada Strings

- `len()`: Fungsi untuk memeriksa panjang string. Contoh sintaks: `len(name)` akan mengembalikan jumlah karakter dalam string.
- Concatenation: Menggabungkan dua string menggunakan operator `+`. Contoh: `a + b` akan menggabungkan string `a` dan `b`.

1.20 Contoh Penggunaan

- Single Line String: `a = 'this is a single line'`
- Multi-Line String: Untuk membuat string yang panjang, gunakan backslash (`\`) di akhir baris. Contoh:

```
b = 'this_is_a_multi-line_string_example_\nthat_continues_here.'
```

- Output: Menggunakan `print()` untuk menampilkan string. Contoh: `print(a)` akan mencetak nilai dari `a`.

1.21 Cheatsheets Tipe Data dan Fungsi

Berikut ini adalah panduan singkat tentang jenis data dalam Python.

1.21.1 Tipe Data

Tipe Data	Meaning	Example
string	Text	'Hello', "Testing 123"
integer	Numbers	-5, 4, 3, 2, 0
float	Decimals	2.4, 5.2, 1000.00

1.21.2 Flow Control

Operator Perbandingan

Operator	Meaning	Example
==	Equals	a == b
!=	Not Equal	a != b
<	Less than	a < b
>	Greater than	a > b
<=	Less than or Equal to	a <= b
>=	Greater than or Equal to	a >= b

1.21.3 Indentasi

Indentasi digunakan untuk mendefinisikan blok kode dalam Python. Berbeda dengan banyak bahasa pemrograman yang menggunakan kurung kurawal untuk mengelompokkan kode, Python menggunakan indentasi.

Indentasi wajib digunakan dalam Python dan digunakan untuk menunjukkan pernyataan mana yang termasuk dalam loop, fungsi, kondisional, dan struktur lainnya.

```
if x > 0: # Colon indicates the start of the block
    print("Positive") # Indented
else:
    print("Non-positive") # Indented
```

1.21.4 Komentar

Menempatkan simbol # di depan teks yang ingin Anda jadikan komentar akan membuat Python mengabaikan semua teks dari titik tersebut hingga akhir baris saat ini.

Triple Quotes (''' or """): Python menggunakan tanda kutip tunggal atau ganda bertiga untuk membuat komentar **multi-baris**. Tanda kutip ini umumnya digunakan untuk docstrings atau komentar yang mencakup beberapa baris.

Simbol # akan membuat Python mengabaikan semua teks mulai dari titik tersebut hingga akhir baris saat ini, sehingga komentar inline dapat dibuat dengan cara ini.

Kolon (:): Digunakan untuk menandai awal blok kode, seperti dalam perulangan, kondisi, dan definisi fungsi.

```
# Single Line comment

# This is a multiline comment
# which can be used for long comments

x = 1 # assigns value of 1 to x

if x > 0: # Colon starts the block
    print("Positive")
```

1.21.5 Built-in Functions

- `print()`

Fungsi ini mencari perangkat output default, terminal Anda, dan menampilkan nilai yang diteruskan kepadanya.

```
print("Hello")
```

- `input()`

Fungsi ini mencari perangkat input default, yaitu keyboard Anda, dan menangkap nilainya. Nilai ini kemudian dapat ditugaskan atau digunakan.

```
print("Where do you live?")
location = input()
print("So you live in " + location)
```

Catatan: Blok kode di atas akan menampilkan prompt kepada pengguna dengan pertanyaan "Di mana Anda tinggal?". Pengguna kemudian akan memasukkan respons dan menekan tombol Return untuk melanjutkan eksekusi kode yang tersisa. Respons yang dimasukkan oleh pengguna disimpan, dikembalikan sebagai output, atau dapat ditugaskan ke variabel tertentu tergantung pada konteks penggunaannya.

- `len()`

Fungsi ini mengembalikan panjang atau jumlah elemen yang terdapat dalam struktur yang diterapkan padanya. Struktur tersebut dapat berupa string, array, daftar, tuple, kamus, atau urutan apa pun.

```
len("Hello")
5
```

- `str()`

Fungsi ini dapat digunakan untuk mengubah nilai yang diberikan menjadi String.

```
str(55)
'55'
```

- `int()`

Fungsi ini dapat digunakan untuk mengubah nilai yang diberikan menjadi bilangan bulat.

```
int('75')
75
```

- `float()`

Fungsi ini dapat digunakan untuk mengubah nilai yang diberikan menjadi bilangan float.

```
some_int = 10
float(some_int)
10.0
```

1.21.6 Membuat Fungsi

Fungsi dalam Python memerlukan kata kunci untuk mendefinisikannya: ‘def’ diikuti oleh identifikasi (nama), yang membentuk tanda tangan fungsi. Tubuh fungsi berisi kode yang akan dijalankan saat fungsi dipanggil.

```
def say_hello():
    return "Hello there!"

# With parameters
def say_hello(you):
    return "Hello " + you
```

1.22 Apa itu Typecasting?

Typecasting adalah proses mengubah satu tipe data menjadi tipe data lain. Dalam Python, ada dua jenis konversi: implicit dan explicit.

1.23 Implicit Type Conversion

Implicit Conversion : Dilakukan secara otomatis oleh compiler Python untuk mencegah kehilangan data. Contoh: mengubah int menjadi float jika nilai yang dimasukkan adalah desimal.

Penting untuk dicatat bahwa Python hanya dapat mengonversi nilai jika tipe data tersebut kompatibel. Misalnya, int dan float kompatibel, tetapi String dan int tidak.

1.24 Explicit Type Conversion

Explicit Conversion : Dilakukan oleh pengembang menggunakan fungsi yang disediakan oleh Python. Beberapa

- `str()`: Mengubah tipe data menjadi string. Sintaks: `str(value)`.
- `int()`: Mengubah tipe data menjadi integer. Sintaks: `int(value)`.
- `float()`: Mengubah tipe data menjadi float. Sintaks: `float(value)`.

1.25 Fungsi Typecasting Lainnya

- `ord()`: Mengembalikan integer yang mewakili karakter unicode yang mendasarinya.
- `hex()`: Mengonversi integer menjadi string heksadesimal.
- `oct()`: Mengambil integer dan mengembalikan string yang mewakili angka oktal.
- `tuple()`, `set()`, `list()`, `dict()`: Fungsi lain untuk mengonversi tipe data.

1.26 Fungsi Input

Fungsi `input()` digunakan untuk mendapatkan data dari pengguna.

Sintaks: `input("Prompt")`

- Contoh: `email = input("Please enter your email address: ")`
- Penjelasan: Pengguna akan diminta untuk memasukkan alamat email, yang kemudian disimpan dalam variabel `email`.

1.27 Fungsi Print

Fungsi `print()` digunakan untuk menampilkan output ke layar.

Sintaks: `print(value1, value2, ..., sep=' ', end='\n')`

- Contoh: `print("Hello", "World", sep=" ")`
- Penjelasan: Menampilkan "Hello, World" dengan koma sebagai pemisah.

1.28 Parameter Tambahan untuk Print

- **sep**: Menentukan pemisah antara objek yang dicetak.
- **end**: Menentukan apa yang dicetak di akhir output (default adalah new-line).
- **file**: Menentukan di mana nilai dicetak (default adalah STD out).
- **flush**: Boolean untuk mengatur apakah buffer harus dibersihkan.

1.29 Contoh Penggunaan Input dan Print

Mengambil dua angka dari pengguna dan mencetak hasil penjumlahannya:

```
num1 = int(input("Please enter the first number: "))
num2 = int(input("Please enter the second number: "))
print("The sum is:", num1 + num2)
```

Penjelasan: Menggunakan `int()` untuk mengonversi input menjadi integer sebelum melakukan penjumlahan.

1.30 Penanganan Tipe Data

Input dari fungsi `input()` selalu berupa string.

Sintaks: `type(variable)`

- Contoh: `print(type(num1))`
- Penjelasan: Menampilkan tipe data dari variabel `num1`, yang akan menunjukkan bahwa itu adalah string jika tidak dikonversi.

1.31 Penggabungan String

Menggabungkan dua atau lebih string menggunakan operator `+`.

Contoh:

```
first_name = input("Please enter your first name: ")
last_name = input("Please enter your last name: ")
print("Hello, " + first_name + " " + last_name)
```

Penjelasan: Menampilkan sapaan dengan nama depan dan belakang yang dimasukkan pengguna.

1.32 Melihat Lebih Dalam ke Type Casting

Ada beberapa situasi di mana tipe data dari suatu nilai perlu diubah menjadi tipe data lain.

Proses ini dikenal sebagai casting tipe.

Cara lain yang lebih informal untuk menyebutnya adalah "konversi tipe data".

Contoh paling sederhana dari konversi data dapat dilihat pada perbandingan berikut:

```
print(10 == 10)
```

Dalam potongan kode di atas, saya bertanya kepada Python apakah angka 10 sama dengan angka 10, dan saya akan mendapatkan nilai boolean `True` yang dicetak, yang mengonfirmasi bahwa memang keduanya sama.

Bagaimana jika saya melakukan ini?

```
print(10 == 10.00)
```

Sekali lagi, nilai boolean `True` ditampilkan di layar.

Sekarang, Anda mungkin keberatan bahwa, ya, 10 secara teknis tidak sama dengan 10.0 - karena, bisa dibilang, angka pertama adalah bilangan bulat, dan angka kedua adalah bilangan desimal. Anda benar - meskipun ini adalah angka yang sama, mereka memiliki tipe data yang berbeda.

Namun, Python di sini melakukan apa yang disebut "konversi tipe implisit".

Untuk memahami bagaimana ini bekerja, saya akan sedikit mengubah contoh sebelumnya. Alih-alih meminta Python untuk membandingkan kedua angka dan mengembalikan apakah keduanya sama atau tidak, saya akan meminta Python untuk menampilkan hasil penjumlahan kedua angka tersebut.

```
print(10 + 10.0)
```

Hasil cetak adalah 20.0.

Hasil ini memungkinkan saya menyimpulkan bahwa saat Python menjalankan operasi yang melibatkan bilangan bulat dan bilangan desimal, ia secara implisit mengubah tipe bilangan bulat menjadi bilangan desimal, lalu menyelesaikan operasi tersebut.

Untuk benar-benar menekankan poin ini, saya dapat memperluas contoh sebelumnya dengan menggunakan fungsi `type()`:

```
print(type(10 + 10.0))
```

Hasilnya adalah `<class 'float'>`, jadi ini mengonfirmasi kesimpulan saya.

Sekarang saya akan menunjukkan kepada Anda program kecil dalam Python yang bekerja dengan angka:

```
user_num_1 = input('First number is: ')
user_num_2 = input('Second number is: ')
user_sum = user_num_1 + user_num_2
print(user_sum)
```

Ketika saya menjalankan program ini, misalnya, saya dapat memasukkan nilai angka pertama sebagai 5, dan nilai angka kedua juga sebagai 5, dengan harapan nilai `user_sum` yang dicetak akan menjadi 10.

Namun, ketika saya melakukan hal itu, angka 55 yang dicetak.

Mengapa ini tidak berfungsi?

Jawabannya cukup sederhana: segala sesuatu yang diketik oleh pengguna oleh Python dikonversi menjadi tipe data string.

Ini berarti, meskipun saya mengetik angka ke dalam dua input tersebut, yang disimpan dalam `user_num_1` dan `user_num_2` sebenarnya adalah string.

Secara efektif, ini sama saja dengan jika saya hanya melakukan ini:

```
user_num_1 = "5"
user_num_2 = "5"
user_sum = user_num_1 + user_num_2
print(user_sum)
```

Kali ini, hasil cetak dari `user_sum` masih "55", tetapi hal ini tidak melanjutkan.

Untuk membuat kode Python saya berfungsi sesuai yang saya inginkan, saya perlu melakukan konversi tipe secara eksplisit.

Dengan kata lain, saya harus mengubah nilai "5" menjadi nilai 5.

Berikut adalah kode saya yang telah diperbarui:

```
user_num_1 = input('First number is: ')
user_num_2 = input('Second number is: ')
user_sum = float(user_num_1) + float(user_num_2)
print(user_sum)
```


Apa yang saya lakukan di sini adalah memastikan bahwa program saya dapat menangani input berupa bilangan float dan tetap berfungsi dengan benar.

Dengan kata lain, saya memastikan bahwa jika pengguna memasukkan nilai float 5.5 sebagai angka pertama dan kedua saat menjalankan kode di atas, outputnya tidak akan menimbulkan kesalahan. Sebaliknya, outputnya akan menjadi 11.0 seperti yang diharapkan.

Bagaimana jika saya memutuskan untuk menampilkan beberapa kata kepada pengguna, memberitahu mereka apa yang terjadi?

Ini adalah upaya untuk melakukannya:

```
num_1 = input('First number is: ')
num_2 = input('Second number is: ')
user_sum = float(num_1) + float(num_2)
print("The sum of " + num_1 + " and " + num_2 + " is " +
      user_sum)
```

Jika saya menjalankan kode di atas, akan muncul kesalahan berikut:

```
Traceback (most recent call last):
File "<string>", line 4, in <module>
TypeError: hanya dapat menggabungkan str (bukan "float")
dengan str
```

Artinya, saya tidak dapat menggabungkan string dan float seperti itu. Dengan kata lain, meskipun konversi tipe implisit Python berfungsi saat menggunakan operator + pada string dan integer, hal ini tidak berlaku untuk string dan float kecuali Anda secara eksplisit mengonversi float menjadi string.

Perlu diperhatikan juga, Anda juga tidak dapat menggabungkan string dan integer. Hal ini terjadi karena Python menerapkan keamanan tipe, artinya ia memerlukan instruksi eksplisit tentang cara menggabungkan tipe data yang berbeda dalam kasus seperti string dengan integer.

Solusi untuk ini adalah melakukan konversi tipe secara eksplisit, sebagai berikut.

```
n1 = input('First number is: ')
n2 = input('Second number is: ')
user_sum = float(n1) + float(n2)
print("The sum of " + n1 + " and " + n2 + " is " + str(
      user_sum))
```

Kali ini, hasilnya adalah: Jumlah dari 5,5 dan 5,5 adalah 11,0.

1.33 Operator Matematika

- Addition (+): Digunakan untuk menjumlahkan angka. Contoh sintaks: 2 + 3 menghasilkan 5.
- Subtraction (-): Digunakan untuk mengurangi angka. Contoh sintaks: 3 - 2 menghasilkan 1.

- Division (/): Digunakan untuk membagi angka. Contoh sintaks: `35 / 5` menghasilkan 7.0.
- Multiplication (*): Digunakan untuk mengalikan angka. Contoh sintaks: `7 * 4` menghasilkan 28.

1.34 Operator Logika

- AND: Memeriksa apakah semua kondisi benar. Contoh: `a < 5 and a < 10` hanya benar jika a lebih besar dari 5 dan kurang dari 10.
- OR: Memeriksa apakah setidaknya satu kondisi benar. Contoh: `a < 5 or b < 10` benar jika salah satu dari a atau b memenuhi kondisi.
- NOT: Mengembalikan nilai false jika hasilnya true. Contoh: `not (a < 5)` akan benar jika a tidak lebih besar dari 5.

1.35 Contoh Penggunaan Operator dalam Pernyataan Kondisional

Operator biasanya digunakan dengan pernyataan kondisional untuk mengontrol alur program. Contoh: Untuk memberikan diskon di restoran, Anda bisa memeriksa apakah pelanggan adalah anggota program loyalitas dan telah menghabiskan lebih dari \$100.

```
if in_loyalty_program and total_bill > 100:
    total_bill = total_bill - (total_bill * 0.1)
```

Contoh Lain:

- Mathematical Operators:
 - `print(2 + 2)` akan menampilkan 4.
 - `print(35 / 5)` akan menampilkan 7.0.
- Logical Operators:
 - `a = True; b = True; if a and b: print("all true")` hanya akan mencetak "all true" jika kedua a dan b adalah true.

1.36 Apa itu Control Flow?

Control Flow merupakan urutan di mana instruksi dalam program dieksekusi. Program akan mengambil tindakan yang berbeda berdasarkan keputusan yang dibuat.

Dalam Python, ada dua jenis control flow:

- Conditional Statements: Seperti `if`, `else`, dan `elif`.
- Loops: Seperti `for` loop dan `while` loop.

1.37 Conditional Statements

- if: Menyatakan bahwa jika kondisi terpenuhi, maka fungsi akan dijalankan.

```
if condition:
    # code to execute
```

- else: Menjalankan fungsi ketika semua kondisi tidak terpenuhi.

```
else:
    # code to execute if the condition is false
```

- elif: Menyatakan bahwa jika kondisi sebelumnya tidak terpenuhi, coba kondisi ini.

```
elif another_condition:
    # code to execute if this condition is true
```

1.38 Contoh Penggunaan Control Flow

Dalam contoh restoran, jika total tagihan (`bill_total`) lebih dari 100, maka diskon diterapkan.

```
bill_total = 114
discount1 = 10
if bill_total > 100:
    bill_total = bill_total - discount1
print("Total bill:", str(bill_total))
```

Untuk menambahkan diskon untuk tagihan di atas 200, gunakan elif,

```
elif bill_total > 200:
    discount2 = 20
    bill_total = bill_total - discount2
```

1.39 Penggunaan Match Statement

Match Statement diperkenalkan di Python versi 3.10, match statement memungkinkan penulisan kode yang lebih bersih dan mudah dibaca dibandingkan dengan if statement.

```
match variable:
    case value1:
        # aksi jika variable sama dengan value1
    case value2:
        # aksi jika variable sama dengan value2
    case _:
        # aksi default jika tidak ada yang cocok
```

1.40 Perbandingan dengan if Statement

If Statement mengharuskan penulisan banyak kondisi dengan if, elif, dan else, yang dapat membuat kode menjadi kompleks.

```
if http_status == 200:
    print("Success")
elif http_status == 404:
    print("Not Found")
else:
    print("Unknown")
```

1.41 Fitur Penting dari Match Statement

- Menggabungkan Beberapa Kondisi: Anda dapat menggunakan operator `or` untuk menggabungkan beberapa kondisi dalam match statement.
- Default Case: Menggunakan `case _` untuk menangani situasi di mana tidak ada nilai yang cocok, mirip dengan else dalam if statement.
- Fleksibilitas: Memberikan cara yang lebih bersih untuk menangani banyak kondisi dibandingkan dengan if statement.

2 Dasar Programming dengan Python

3 Paradigma Pemrograman

4 Modul, Paket, Pustaka, and Alat-alat